# CS 374: Section A - Homework 7, Problem 2

Anirudh Konidala

October 20, 2026

Rectangle $r$ has lower-left corner $(A[r], B[r])$ and upper-right corner $(C[r], D[r])$. All $4n$ input numbers are distinct, so $A[r] < C[r]$ and $B[r] < D[r]$ for every rectangle $r$.

**Coordinate arrays:** Sort the $2n$ $x$-coordinates $A[1..n]$ and $C[1..n]$ into an array $X[1..2n]$, and sort the $2n$ $y$-coordinates $B[1..n]$ and $D[1..n]$ into an array $Y[1..2n]$. Both arrays are strictly increasing.

**Boxes:** For $1 \le a \le b \le 2n$ and $1 \le c \le d \le 2n$, let $Inside(a, b, c, d)$ be the set of input rectangles $r$ that lie in the box $[X[a], X[b]] \times [Y[c], Y[d]]$, that is:

$$X[a] \le A[r], \quad C[r] \le X[b], \quad Y[c] \le B[r], \quad D[r] \le Y[d].$$

This is a helper set, not a recursive function; the algorithm tests membership on the fly.

## Recursive function

For $1 \le a \le b \le 2n$ and $1 \le c \le d \le 2n$, define:

$$\boxed{MaxSep(a, b, c, d) = \text{the maximum size of an orthogonally separable subset of } Inside(a, b, c, d).}$$

The box $[X[1], X[2n]] \times [Y[1], Y[2n]]$ contains every input rectangle, so we need $MaxSep(1, 2n, 1, 2n)$.

## Recurrence

$$MaxSep(a, b, c, d) = \max \begin{cases} 0 & \\ 1 & \text{if } Inside(a, b, c, d) \neq \varnothing \\ \max\limits_{a \le p < b} \left( MaxSep(a, p, c, d) + MaxSep(p+1, b, c, d) \right) & \text{(vertical cut)} \\ \max\limits_{c \le q < d} \left( MaxSep(a, b, c, q) + MaxSep(a, b, q+1, d) \right) & \text{(horizontal cut)} \end{cases}$$

A maximum over an empty range of $p$ or $q$ is left out.

**Base case:** If $a = b$ and $c = d$, there is no cut. No rectangle lies in the box, because every rectangle has $A[r] < C[r]$. So the recurrence gives $MaxSep(a, a, c, c) = 0$, which is correct.

**Idea:** A vertical cut between $X[p]$ and $X[p+1]$ splits the box into a left box $[X[a], X[p]] \times [Y[c], Y[d]]$ and a right box $[X[p+1], X[b]] \times [Y[c], Y[d]]$. A rectangle that crosses the cut is in neither box, and it cannot be in a subset that this cut separates. Horizontal cuts are the same, with $y$ in place of $x$.

**Every option is possible:** The empty set and every one-element subset are orthogonally separable. For a vertical cut at index $p$, let $T_1$ be an orthogonally separable subset of $\mathit{Inside}(a, p, c, d)$ and $T_2$ an orthogonally separable subset of $\mathit{Inside}(p+1, b, c, d)$, both of maximum size. Every rectangle in $T_1$ has $C[r] \le X[p]$, and every rectangle in $T_2$ has $A[r] \ge X[p+1]$. So the vertical line $x = (X[p] + X[p+1])/2$ has $T_1$ on its left and $T_2$ on its right, and $T_1$ and $T_2$ are disjoint. If both are nonempty, $T_1 \cup T_2$ is orthogonally separable by the definition. If one of them is empty, $T_1 \cup T_2$ is the other one, which is also orthogonally separable. In both cases $T_1 \cup T_2 \subseteq \mathit{Inside}(a, b, c, d)$ has size $\mathit{MaxSep}(a, p, c, d) + \mathit{MaxSep}(p+1, b, c, d)$. Horizontal cuts are the same.

**The best option is at least the answer:** Let $T$ be an orthogonally separable subset of $\mathit{Inside}(a, b, c, d)$ of maximum size. We use the following cases:

- **$|T| = 0$ or $|T| = 1$:** The first two options give $|T|$.
- **$|T| \ge 2$ and a vertical line separates $T$:** By the definition, there is a line $x = t$ and a partition of $T$ into nonempty orthogonally separable sets $T_1$ and $T_2$, with $C[r] \le t$ for every $r \in T_1$ and $A[r] \ge t$ for every $r \in T_2$. Let $t^* = \max_{r \in T_1} C[r]$, and let $p$ be the index with $X[p] = t^*$. Every $r \in T_2$ has $A[r] \ge t \ge t^*$, and $A[r] \ne t^*$ because all input numbers are distinct, so $A[r] > X[p]$, that is, $A[r] \ge X[p+1]$. This gives $T_2 \subseteq \mathit{Inside}(p+1, b, c, d)$. Every $r \in T_1$ has $C[r] \le X[p]$, which gives $T_1 \subseteq \mathit{Inside}(a, p, c, d)$. The index $p$ is in range: for the rectangle $r_1 \in T_1$ with $C[r_1] = t^*$ we have $X[a] \le A[r_1] < X[p]$, so $p > a$; for any $r_2 \in T_2$ we have $X[p] < A[r_2] < C[r_2] \le X[b]$, so $p < b$. So $|T| = |T_1| + |T_2| \le \mathit{MaxSep}(a, p, c, d) + \mathit{MaxSep}(p+1, b, c, d)$.
- **$|T| \ge 2$ and a horizontal line separates $T$:** The same argument with $Y$, $B$ and $D$ in place of $X$, $A$ and $C$ gives an index $c \le q < d$ with $|T| \le \mathit{MaxSep}(a, b, c, q) + \mathit{MaxSep}(a, b, q+1, d)$.

So the maximum of the options is exactly $\mathit{MaxSep}(a, b, c, d)$.

## Memoization and evaluation order

We store $\mathit{MaxSep}$ in a four-dimensional array $\mathit{MaxSep}[1..2n, 1..2n, 1..2n, 1..2n]$ and use only the entries with $a \le b$ and $c \le d$.

Every entry depends only on entries with a larger $a$, or the same $a$ and a smaller $b$, or the same $(a, b)$ and a larger $c$, or the same $(a, b, c)$ and a smaller $d$. So we use four nested loops, from the outside in: $a$ decreases from $2n$ to 1, $b$ increases from $a$ to $2n$, $c$ decreases from $2n$ to 1, and $d$ increases from $c$ to $2n$.

```
MaxSeparable(A[1..n], B[1..n], C[1..n], D[1..n]):
  X[1..2n] ← sorted values of A[1..n] and C[1..n]
  Y[1..2n] ← sorted values of B[1..n] and D[1..n]
  for a ← 2n down to 1
    for b ← a to 2n
      for c ← 2n down to 1
        for d ← c to 2n
          v ← 0
          for r ← 1 to n            ⟨⟨is Inside(a, b, c, d) nonempty?⟩⟩
            if X[a] ≤ A[r] and C[r] ≤ X[b] and Y[c] ≤ B[r] and D[r] ≤ Y[d]
              v ← 1
          for p ← a to b − 1        ⟨⟨vertical cuts⟩⟩
            v ← max{v, MaxSep[a, p, c, d] + MaxSep[p + 1, b, c, d]}
          for q ← c to d − 1        ⟨⟨horizontal cuts⟩⟩
            v ← max{v, MaxSep[a, b, c, q] + MaxSep[a, b, q + 1, d]}
          MaxSep[a, b, c, d] ← v
  return MaxSep[1, 2n, 1, 2n]
```

## Running time

Sorting takes $O(n \log n)$ time. The array has at most $(2n)^4 = O(n^4)$ used entries. For each entry, the test for a rectangle in the box takes $O(n)$ time, and the two loops over cuts try at most $4n$ indices in $O(1)$ time each. So each entry takes $O(n)$ time, and the algorithm runs in

$$\boxed{O(n^5) \text{ time.}}$$

**Sources and collaborators:** None.

**LLM use:** I used Claude (Anthropic) as a tutor for this problem: it helped me understand the problem and guided me, through explanation and questions, toward a correct approach. I wrote and typeset this entire submission myself.